

# Semafori

Alessia Smeralda Brunello

Matricola: 544964

## **Hands-On 12**

# Indice

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduzione</b>                                | <b>2</b> |
| <b>2</b> | <b>Definizione del problema</b>                    | <b>2</b> |
| 2.1      | Barriera di sincronizzazione . . . . .             | 2        |
| 2.2      | Produttore-consumatore con buffer finito . . . . . | 2        |
| <b>3</b> | <b>Metodologia</b>                                 | <b>3</b> |
| 3.1      | Barriera con semafori . . . . .                    | 3        |
| 3.2      | Produttore-consumatore . . . . .                   | 4        |
| <b>4</b> | <b>Presentazione dei risultati</b>                 | <b>5</b> |
| 4.1      | Risultati della barriera . . . . .                 | 5        |
| 4.2      | Risultati del produttore-consumatore . . . . .     | 6        |
| <b>5</b> | <b>Conclusioni</b>                                 | <b>6</b> |

# 1 Introduzione

Il presente hands-on riguarda l'utilizzo dei semafori per la sincronizzazione tra thread. I semafori sono primitive di sincronizzazione che permettono di controllare l'accesso a risorse condivise e di coordinare l'esecuzione di più flussi concorrenti.

L'esercizio è stato suddiviso in due parti principali:

- implementazione di una barriera di sincronizzazione per N thread;
- implementazione del problema produttore-consumatore con buffer finito.

La soluzione è stata sviluppata in C utilizzando i thread POSIX e i semafori POSIX. Il codice è stato organizzato secondo un approccio modulare, separando le funzionalità principali in file distinti e utilizzando un `Makefile` per automatizzare la compilazione.

## 2 Definizione del problema

### 2.1 Barriera di sincronizzazione

Il primo problema richiede la realizzazione di un meccanismo di barriera che consenta a N thread di partire tutti nello stesso momento. Ogni thread può arrivare alla barriera in un istante diverso, ma nessuno deve superarla finché tutti gli altri thread non l'hanno raggiunta.

Formalmente, la barriera rappresenta un punto di sincronizzazione globale: se anche un solo thread non è ancora arrivato, tutti gli altri devono rimanere bloccati.

### 2.2 Produttore-consumatore con buffer finito

Il secondo problema consiste nella realizzazione di un sistema produttore-consumatore con buffer condiviso di dimensione limitata. Il produttore inserisce elementi nel buffer, mentre il consumatore li rimuove.

Poiché il buffer ha dimensione finita:

- il produttore deve attendere se il buffer è pieno;
- il consumatore deve attendere se il buffer è vuoto;
- entrambi devono accedere al buffer in mutua esclusione.

## 3 Metodologia

### 3.1 Barriera con semafori

La barriera è stata implementata tramite una struttura dati contenente:

- **n**, cioè il numero totale di thread attesi;
- **arrived**, cioè il numero di thread arrivati alla barriera;
- **mutex**, semaforo binario usato per proteggere il contatore condiviso;
- **turnstile**, semaforo usato per bloccare i thread fino all'arrivo dell'ultimo.

Il semaforo **mutex** viene inizializzato a 1, poiché deve consentire l'accesso esclusivo alla variabile condivisa **arrived**.

Il semaforo **turnstile**, invece, viene inizializzato a 0, in modo che i thread rimangano bloccati fino all'apertura della barriera.

Quando un thread arriva alla barriera, incrementa il contatore **arrived**. Se il valore del contatore diventa uguale a **n**, significa che il thread corrente è l'ultimo ad arrivare. In questo caso, esso effettua **N** operazioni di **sem\_post** sul semaforo **turnstile**, consentendo a tutti i thread di superare la barriera.

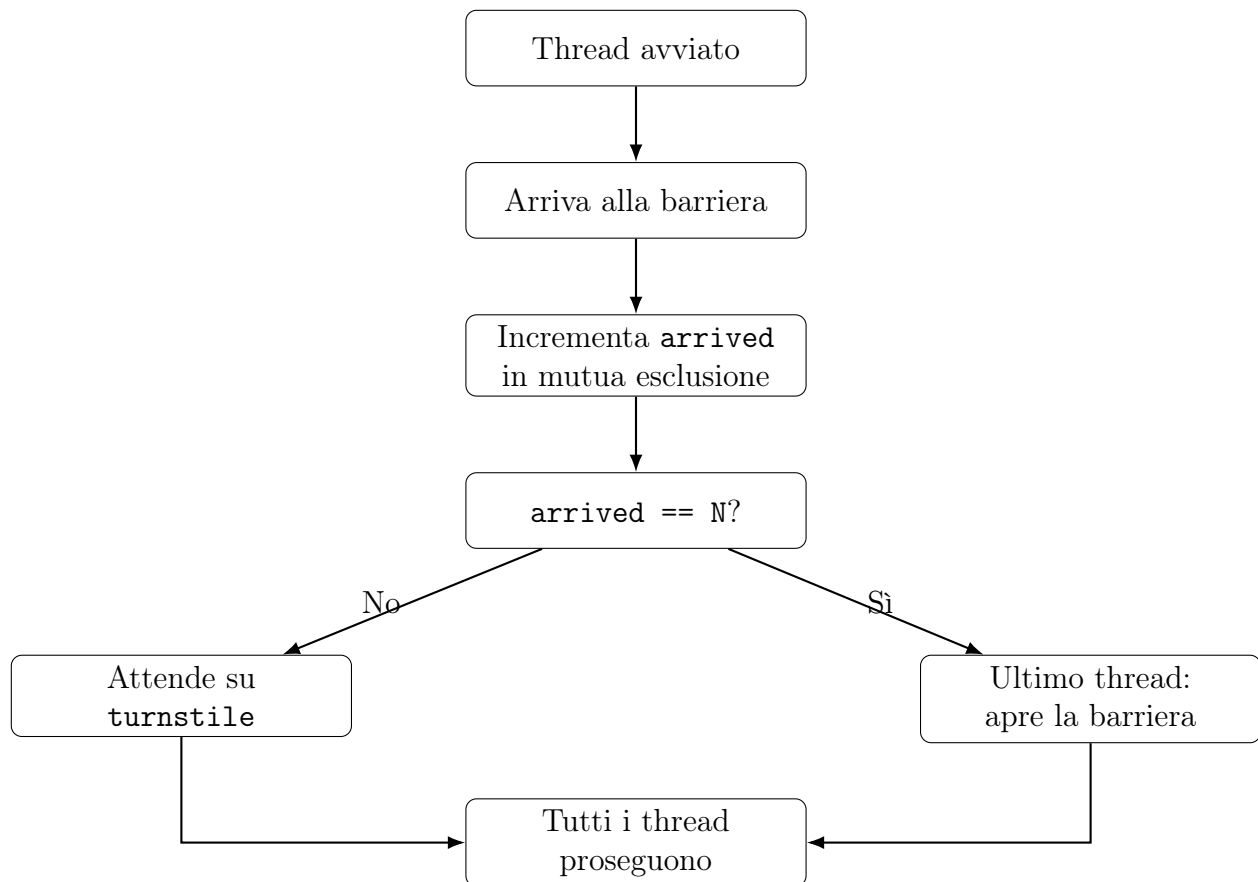


Figura 1: Schema logico della barriera di sincronizzazione.

## 3.2 Produttore-consumatore

Per il problema produttore-consumatore sono stati utilizzati tre semafori:

- `empty` (inizializzato a `BUFFER_SIZE`), rappresenta il numero di posizioni libere nel buffer;
- `full` (inizializzato a 0), rappresenta il numero di elementi disponibili nel buffer;
- `mutex` (inizializzato a 1), protegge l'accesso concorrente al buffer.

Il produttore, prima di inserire un elemento, deve attendere che esista almeno una posizione libera nel buffer. Per questo motivo esegue una `sem_wait(empty)`. Dopo aver inserito un elemento, segnala al consumatore che un nuovo elemento è disponibile tramite `sem_post(full)`.

Il consumatore, invece, prima di rimuovere un elemento, deve attendere che il buffer contenga almeno un elemento. Per questo motivo esegue una `sem_wait(full)`. Dopo aver consumato un elemento, segnala al produttore che si è liberata una posizione nel buffer tramite `sem_post(empty)`.

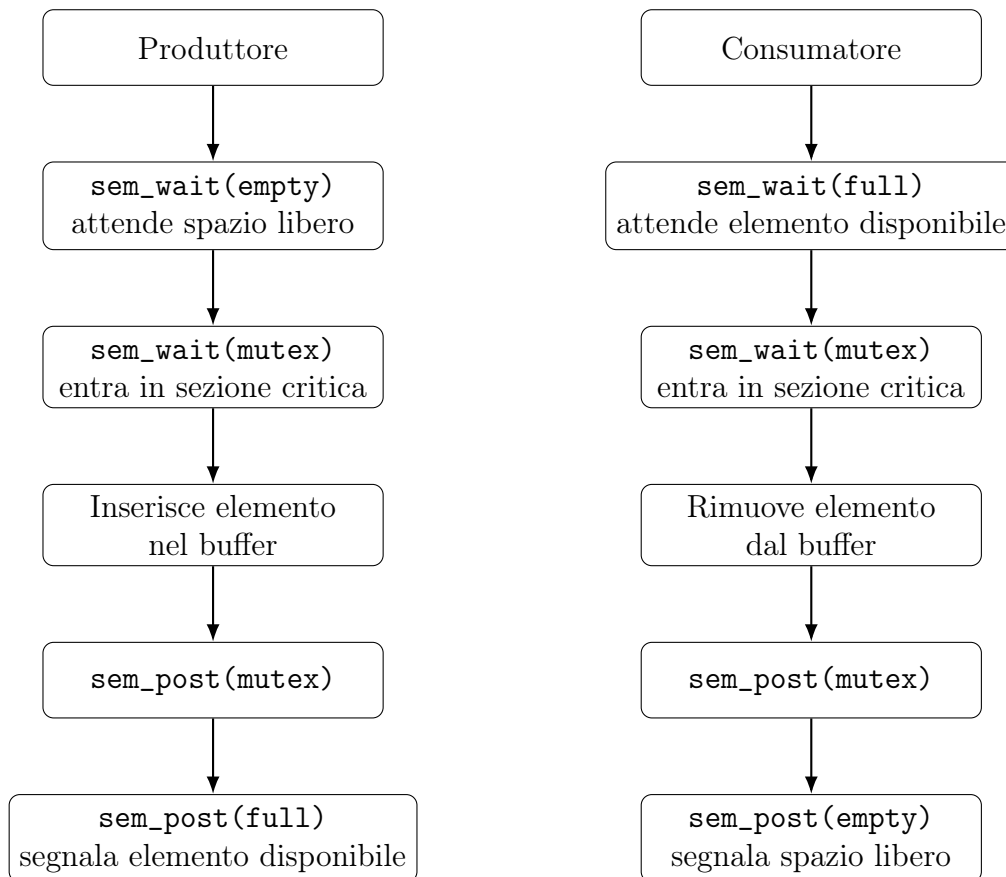


Figura 2: Logica dei semafori nel problema produttore-consumatore.

## 4 Presentazione dei risultati

### 4.1 Risultati della barriera

Eseguendo il programma della barriera con cinque thread, si ottiene il seguente output:



```
alessia23@aleb23:~/hands-on12$ ./barrier 5
Thread 1 pronto
Thread 4 pronto
Thread 3 pronto
Thread 2 pronto
Thread 5 pronto
Ultimo thread ARRIVATO --> Apro la barriera

Thread 2 partito!
Thread 3 partito!
Thread 4 partito!
Thread 1 partito!
Thread 5 partito!
```

Figura 3: Output della barriera

L'ordine con cui i thread stampano il messaggio **pronto** non è deterministico, poiché dipende dallo scheduler del sistema operativo. Tuttavia, il risultato rilevante è che nessun thread stampa **partito** prima che tutti abbiano raggiunto la barriera.

Anche l'ordine dei messaggi **partito** può variare, perché dopo l'apertura della barriera i thread vengono schedulati dal sistema operativo in modo non predicibile. Questo comportamento è corretto: la barriera non impone un ordine di esecuzione, ma garantisce soltanto che tutti i thread superino il punto di sincronizzazione dopo l'arrivo dell'ultimo.

## 4.2 Risultati del produttore-consumatore

L'output del programma produttore-consumatore è il seguente:

A terminal window showing the output of a program. The prompt is 'alessia23@aleb23:~/hands-on12\$ ./prod\_cons'. The output consists of 20 lines, alternating between 'Consumato: ' and 'Prodotto: ' followed by a number from 1 to 10. The sequence is: Consumato: 1, Prodotto: 1, Prodotto: 2, Consumato: 2, Prodotto: 3, Prodotto: 4, Consumato: 3, Prodotto: 5, Prodotto: 6, Consumato: 4, Prodotto: 7, Prodotto: 8, Consumato: 5, Prodotto: 9, Prodotto: 10, Consumato: 6, Consumato: 7, Consumato: 8, Consumato: 9, Consumato: 10.

```
alessia23@aleb23:~/hands-on12$ ./prod_cons
Consumato: 1
Prodotto: 1
Prodotto: 2
Consumato: 2
Prodotto: 3
Prodotto: 4
Consumato: 3
Prodotto: 5
Prodotto: 6
Consumato: 4
Prodotto: 7
Prodotto: 8
Consumato: 5
Prodotto: 9
Prodotto: 10
Consumato: 6
Consumato: 7
Consumato: 8
Consumato: 9
Consumato: 10
```

Figura 4: Output del produttore-consumatore

L'output mostra l'alternanza tra produzione e consumo. Poiché il produttore e il consumatore hanno tempi di esecuzione differenti, è possibile che il produttore inserisca più elementi prima che il consumatore li rimuova. Tuttavia, grazie ai semafori, il produttore non può superare la capacità massima del buffer e il consumatore non può leggere da un buffer vuoto.

La correttezza della soluzione dipende dalla seguente interpretazione dei semafori:

- il produttore attende **empty** quando il buffer non ha spazi liberi;
- il produttore segnala **full** dopo aver inserito un elemento;
- il consumatore attende **full** quando il buffer è vuoto;
- il consumatore segnala **empty** dopo aver rimosso un elemento.

## 5 Conclusioni

L'esperienza ha evidenziato l'importanza dei semafori nella programmazione concorrente.

Nel primo esercizio, i semafori sono stati utilizzati per realizzare una barriera che impedisce ai thread di proseguire finché tutti non hanno raggiunto lo stesso punto di esecuzione.

Nel secondo esercizio, i semafori hanno permesso di coordinare correttamente produttore e consumatore su un buffer condiviso di dimensione finita.

In entrambi i casi, l'utilizzo del semaforo `mutex` ha garantito la mutua esclusione nell'accesso alle variabili condivise, mentre gli altri semafori hanno svolto il ruolo di meccanismi di segnalazione tra thread.

La struttura modulare del codice ha inoltre favorito una separazione chiara tra la logica di sincronizzazione e il codice principale di esecuzione. L'uso del `Makefile` ha permesso di automatizzare la compilazione, rendendo il progetto più ordinato, riutilizzabile e coerente con le buone pratiche di sviluppo.